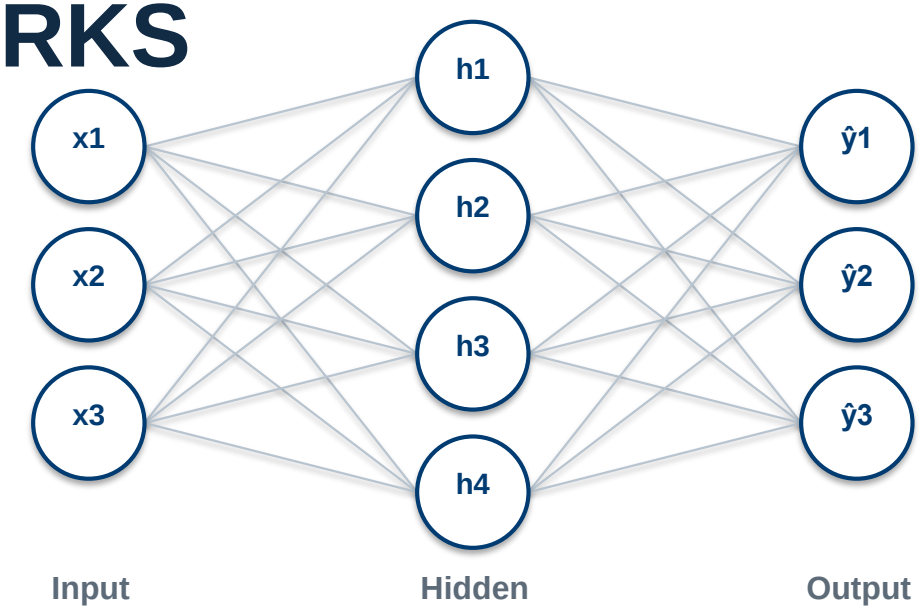
ANKARA UNIVERSITY

COM4573

ARTIFICIAL NEURAL NETWORKS

Week 3 • Multilayer Perceptrons



Fall Semester 2026-2027

Instructor: Dr. Hilal Akarkamçı

Faculty of Engineering • Department of Computer Engineering

Question of the week

From one linear boundary to a network that can build nonlinear decisions.

How can several simple neurons solve a problem that one perceptron cannot?

Think back to XOR: no single straight line works. What could a hidden layer change?

PAIR DISCUSSION • 2 MINUTES

Sketch a possible strategy using more than one linear boundary.

From Week 2 to Week 3

ONE PERCEPTRON

Computes $w^T x + b$ and creates one linear decision boundary.

THE LIMIT

XOR is not linearly separable, so one boundary is insufficient.

THE NEXT IDEA

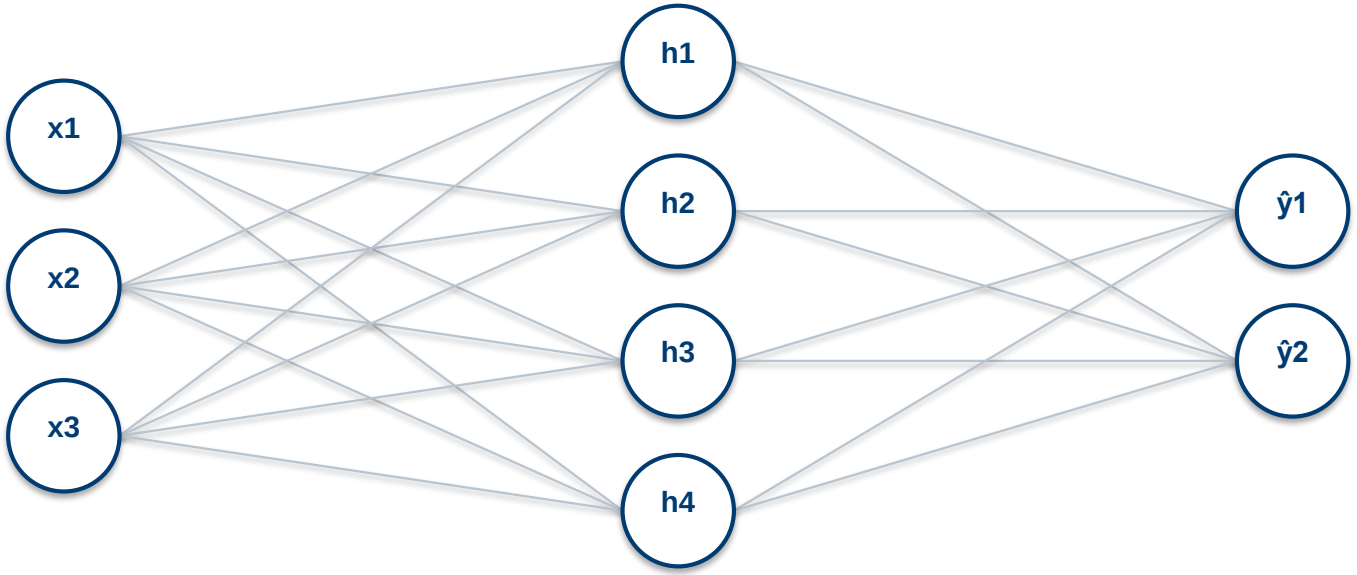
Combine multiple neurons, then combine their outputs.

Today: architecture • hidden layers • forward propagation • nonlinear decision regions • parameter counting

What is a multilayer perceptron?

- A multilayer perceptron (MLP) is a feedforward neural network with an input layer, one or more hidden layers, and an output layer.
- Each neuron forms a weighted sum, adds a bias, and applies an activation function.
- Information moves from inputs toward outputs during the forward pass.
- Nonlinear activation functions make the multilayer composition more expressive than a single linear classifier.

An MLP at a glance



Connections carry learned weights. Each non-input neuron also has a bias.

What does each layer do?

INPUT LAYER

Receives the feature vector x. It does not usually perform the same weighted computation as hidden neurons.

HIDDEN LAYER

Transforms the inputs into intermediate representations. Nonlinear activations allow useful feature combinations.

OUTPUT LAYER

Produces the final task-specific output, such as a class score, probability, or regression value.

From scalar neuron to vector layer

Single neuron

$$z = w^T x + b$$

$$h = f(z)$$

Whole hidden layer

$$z^{(1)} = W^{(1)} x + b^{(1)}$$

$$h^{(1)} = f(z^{(1)})$$

W stores many neuron weights at once; b stores one bias per neuron.

Forward propagation: the central computation

1 • INPUT

Start with x.

2 • LINEAR

$z^1 = W^1x + b^1$

3 • ACTIVATE

$h^1 = f(z^1)$

4 • OUTPUT

$z^2 = W^2h^1 + b^2$

x → hidden pre-activation → hidden activation → output computation → prediction

Forward propagation computes a prediction using the current parameter values. Learning those parameters comes later.

Forward pass by hand • One hidden layer

Input: x = [1, 2]

Hidden neuron 1

$z_1 = 0.5(1) + 0.2(2) - 0.1 = 0.8$

$h_1 = \text{ReLU}(0.8) = 0.8$

Hidden neuron 2

$z_2 = -0.4(1) + 0.6(2) + 0.2 = 1.0$

$h_2 = \text{ReLU}(1.0) = 1.0$

Hidden representation: h = [0.8, 1.0]

Next, the output layer uses h as its input.

Continue the forward pass • Output neuron

Suppose output weights are [1.2, -0.5] and output bias is 0.1.

$$z(out) = 1.2(0.8) - 0.5(1.0) + 0.1 = 0.56$$

If the output activation is sigmoid:

$$\hat{y} = \sigma(0.56) \approx 0.636$$

Interpretation depends on the task and output design. We study activation and loss choices in detail next week.

Your turn • Compute one hidden neuron

$$x = [2, -1] \quad w = [0.4, 0.7] \quad b = 0.3$$

1. Compute $z = w^T x + b$.
2. Apply ReLU: $h = \max(0, z)$.
3. Predict: will this neuron be active?

Checkpoint answer

$$z = 0.4(2) + 0.7(-1) + 0.3 = 0.4$$

$$h = \text{ReLU}(0.4) = 0.4$$

Because $z > 0$, the ReLU unit is active and passes a positive value forward.

Why do hidden layers need nonlinear activation?

If every layer is linear:

$$W^2(W^1x + b^1) + b^2 = W^*x + b^*$$

Stacking linear transformations still gives one linear transformation.

A nonlinear activation between layers prevents this collapse and allows curved or piecewise decision boundaries.

Return to XOR

THE PROBLEM

Positive cases: (0,1) and (1,0)
Negative cases: (0,0) and (1,1)

No single straight line separates the two classes.

Can two hidden neurons create two useful intermediate tests?

That is the key representational idea behind the classic XOR solution.

XOR as hidden features

One conceptual decomposition

HIDDEN UNIT A

Detect a region that includes one positive corner.

HIDDEN UNIT B

Detect another region that includes the other positive corner.

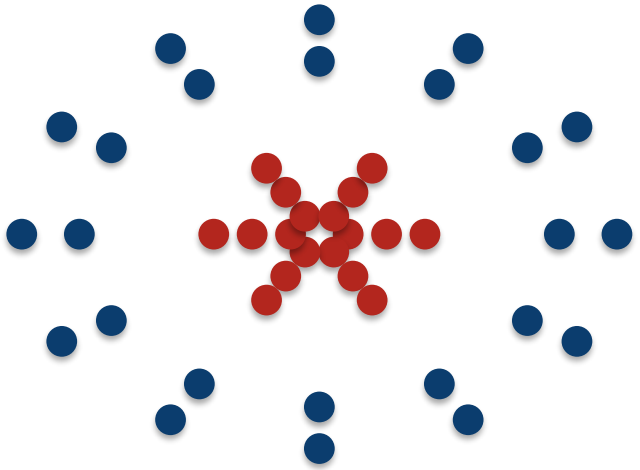
OUTPUT UNIT

Combine hidden responses into the final XOR decision.

Hidden units can transform the original coordinates into a representation where the final decision is easier.

Beyond XOR • Concentric circles

Can one straight line separate the center from the ring?



Class question: How many boundaries might we combine?

CLASS 0 • INNER REGION

Samples near the center belong to one class.

CLASS 1 • OUTER RING

Samples surrounding the center belong to the other class.

WHY A PERCEPTRON FAILS

One line divides the plane into only two half-spaces. It cannot create a closed region around the center.

WHY AN MLP HELPS

Several hidden neurons can combine multiple linear boundaries. Their outputs can form a nonlinear, enclosed decision region.

How can an MLP build an enclosed decision region?

1 • HIDDEN NEURONS

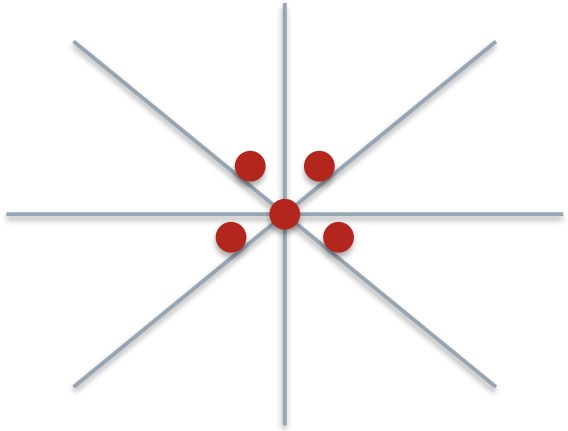
Each hidden neuron can respond to one side of a linear boundary.

2 • COMBINE FEATURES

The hidden layer creates a new representation from several boundary tests.

3 • OUTPUT DECISION

The output neuron combines those hidden features to identify the inner region.



Multiple linear tests → nonlinear region

Key idea: the individual hidden units can be simple; their composition gives the network a more expressive decision boundary.

Hidden layers learn intermediate representations

RAW INPUT

Original measurements or encoded features.

HIDDEN REPRESENTATION

Learned combinations that may expose useful patterns.

TASK OUTPUT

Evidence is combined for classification or regression.

The hidden representation is learned, not manually specified.

Width and depth

WIDTH

Number of neurons in a layer.

More width can provide more parallel features, but also increases parameters and computation.

DEPTH

Number of successive trainable layers.

More depth allows repeated transformations, but can make optimization and model selection harder.

Bigger is not automatically better.

How many parameters does a dense layer have?

For n inputs and m neurons:

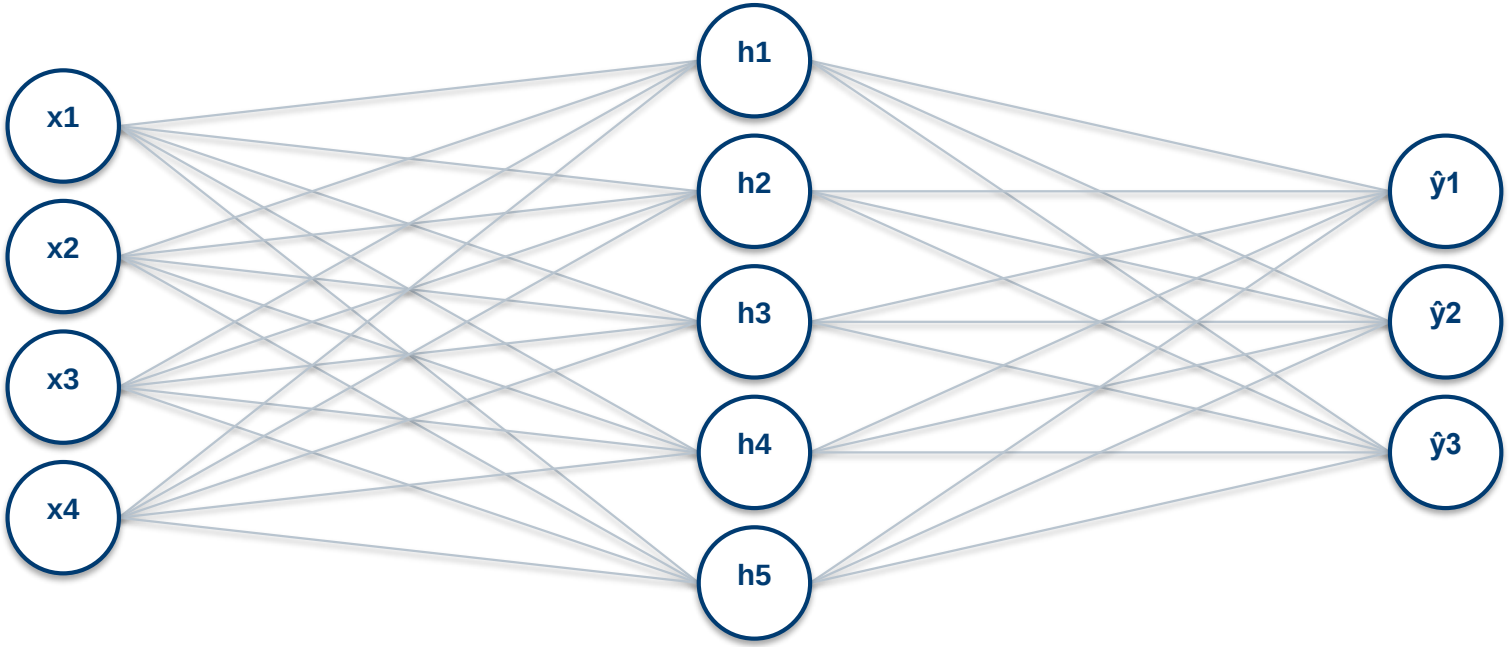
$$\text{weights} = n \times m \qquad \text{biases} = m$$

$$\text{total parameters} = n \times m + m = (n + 1)m$$

EXAMPLE

4 inputs → 8 hidden neurons: 4×8 + 8 = 40 parameters

Parameter counting • Whole network



Layer 1: 4×5 + 5 = 25

Layer 2: 5×3 + 3 = 18

Total = 43 trainable parameters

4 inputs **5 hidden** **3 outputs**

Quick challenge • Count the parameters

Architecture: 6 inputs → 4 hidden neurons → 2 output neurons

How many trainable weights?
How many biases?
How many parameters in total?

Think first. Then explain your counting rule to a partner.

Quick challenge • Answer

Layer 1: 6×4 weights + 4 biases = 28

Layer 2: 4×2 weights + 2 biases = 10

Total = 38 trainable parameters

Binary, multiclass, regression • Output shape

BINARY CLASSIFICATION

Often one output unit. A sigmoid can map a score to a value between 0 and 1.

MULTICLASS

Often one output per class. Softmax can convert scores into a probability distribution.

REGRESSION

One or more numerical outputs. The output activation depends on the target range.

Detailed activation and loss choices are Week 4 topics.

Softmax • A first look

Suppose the network produces three class scores:

$$z = [2.0, 1.0, 0.2]$$

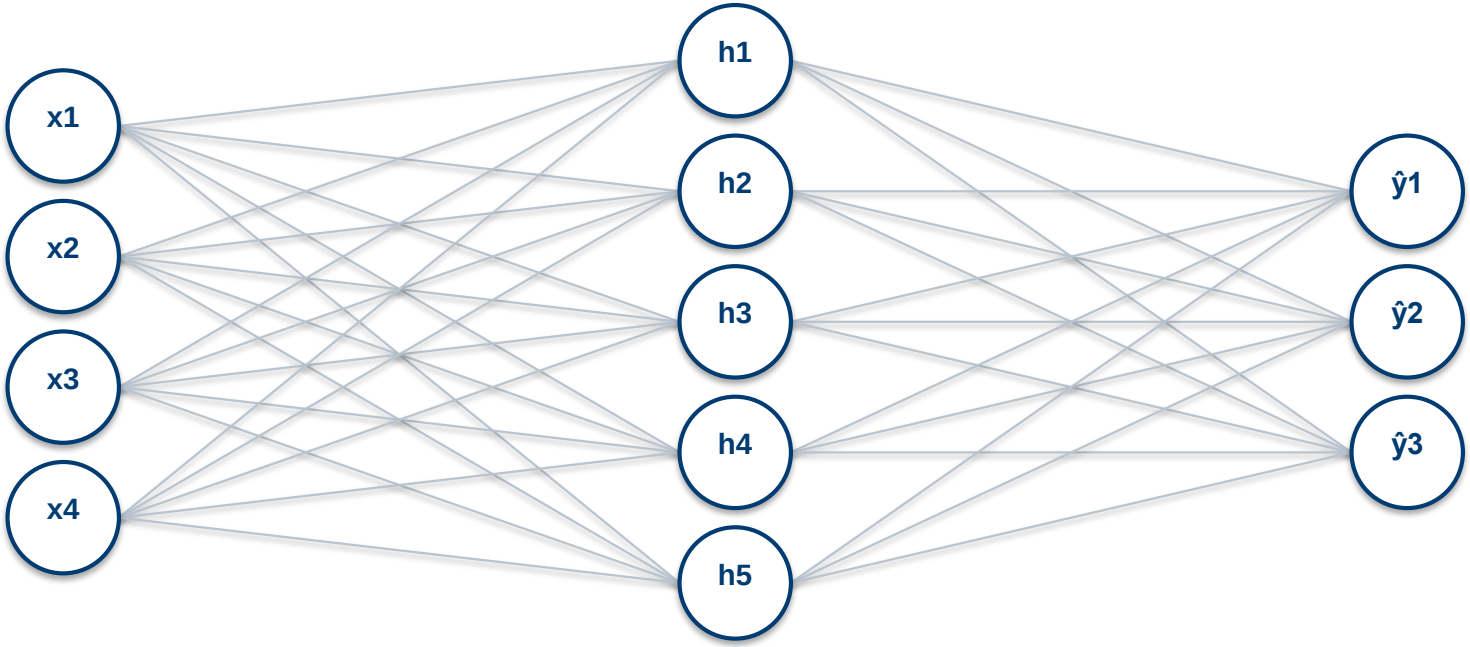
$$\text{softmax}(z) \approx [0.652, 0.240, 0.108]$$

The values are positive and sum to 1. The largest probability gives the predicted class.

Connect to our Week 2 Iris demo

- Week 2 used a perceptron to make the geometry of a linear boundary visible.
- The full Iris dataset has 4 input features and 3 target classes.
- Our Week 2 preview used an MLP with 4 inputs, 8 ReLU hidden units, and 3 softmax outputs.
- Week 3 opens that network and explains what its layers compute.

The Week 2 preview network



4 Iris features

8 hidden units*

3 classes

*Diagram shows 5 hidden circles for readability; the code uses 8 hidden units.

Dense(8, activation="relu") → Dense(3, activation="softmax")

Read the architecture from code

```
ann = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(8, activation="relu"),
    tf.keras.layers.Dense(3, activation="softmax")
])
```

ASK 1

Why is input shape 4?

ASK 2

Why are there 3 outputs?

ASK 3

What does 8 control?

Code reading • Suggested answers

INPUT = 4

Iris has four measured features: sepal length, sepal width, petal length, petal width.

HIDDEN = 8

This is an architecture choice: the hidden layer contains eight trainable neurons.

OUTPUT = 3

There are three target classes: setosa, versicolor, and virginica.

Colab activity • Inspect the MLP

- Reuse the Week 2 Iris notebook and run the 3-class ANN section.
- Print ann.summary() and identify each layer, output shape, and parameter count.
- Before training, ask students to predict the total number of parameters.
- After training, compare test predictions with the true labels and the confusion matrix.

Prediction before execution makes the demo interactive rather than purely observational.

What should ann.summary() reveal?

DENSE 1

Input dimension = 4
Units = 8
Parameters = 4×8 + 8 = 40

DENSE 2

Input dimension = 8
Units = 3
Parameters = 8×3 + 3 = 27

TOTAL

40 + 27 = 67 trainable parameters

The model is small, but every parameter participates in the forward computation.

Forward pass vs learning

FORWARD PASS

Uses current weights and biases to compute predictions.

Today we can follow this computation explicitly.

LEARNING

Changes weights and biases so future predictions improve.

Backpropagation explains how gradients are computed. That is Week 5.

Do not confuse “the network computes” with “the network learns.”

What changes geometrically?

ONE PERCEPTRON

One linear boundary divides the space into two half-spaces.

MULTIPLE HIDDEN UNITS

Several linear tests can partition the input space into multiple regions.

MLP OUTPUT

The output layer combines those regions into a more complex decision function.

Human neural network • Forward pass

Form groups of 5 students.

2 STUDENTS

Act as input features x_1 and x_2.

2 STUDENTS

Act as hidden neurons. Compute z and ReLU.

1 STUDENT

Act as output neuron. Combine hidden values.

THE CLASS

Check each arithmetic step and explain what information moved forward.

Goal: make “forward propagation” a sequence of concrete computations.

Common misconceptions

“MORE LAYERS = BETTER”

No. Extra capacity can be unnecessary and can make training or generalization harder.

“HIDDEN = UNKNOWN MAGIC”

No. A hidden unit performs a defined weighted computation and activation.

“FORWARD PASS = TRAINING”

No. Forward propagation computes outputs; training additionally updates parameters.

How do we choose an MLP architecture?

- Start from the task: input representation and required output determine the first and last dimensions.
- Hidden width and depth are model-design choices, not values directly given by the dataset.
- Use validation performance and problem constraints to compare reasonable architectures.
- Consider parameter count, overfitting risk, computation, and interpretability alongside accuracy.

Architecture choice • Mini discussion

Problem: classify machine condition from 20 sensor features into 4 fault classes.

Discuss:

- What must the input dimension be?
- What must the output dimension be?
- Would 5, 50, or 5000 hidden units all be equally sensible starting points? Why?
- What evidence would you use to compare architectures?

Capacity and generalization

TOO SIMPLE

May underfit because it cannot represent the relevant relationship.

APPROPRIATE CAPACITY

Can capture useful structure while still generalizing to unseen data.

TOO FLEXIBLE

May fit training details that do not transfer to new data.

Architecture is one source of model capacity. Regularization is a later topic.

What does a hidden neuron “mean”?

- A hidden neuron computes a learned feature, but that feature does not always have a simple human-readable label.
- Its response depends on incoming weights, bias, activation, and the previous layer representation.
- Individual units can be informative, but network behavior often depends on distributed combinations of units.
- Interpretability requires evidence, not just assigning intuitive names to neurons.

Perceptron vs MLP

PERCEPTRON

Single trainable decision unit
Linear decision boundary
Classic binary form
Simple parameter interpretation

MLP

Multiple layers of trainable units
Nonlinear decision functions with nonlinear activations
Flexible output structures
More parameters and design choices

Back to the question of the week

How can several simple neurons solve a problem that one perceptron cannot?

Hidden neurons create intermediate nonlinear representations. The output layer combines those representations, allowing the network to form decision functions that are not restricted to one straight line or hyperplane.

MLP = composition of learned transformations

Week 3 • Key takeaways

- An MLP organizes neurons into input, hidden, and output layers.
- Forward propagation moves information through weighted sums, biases, and activation functions.
- Nonlinear activations allow multilayer networks to represent nonlinear relationships.
- Hidden layers learn intermediate representations that can make a task easier for later layers.
- Dense-layer parameter count is inputs × units + units.
- Architecture controls capacity, but larger networks are not automatically better.

Questions 1 to 5

- Define a multilayer perceptron and distinguish it from a single perceptron.
- Explain the roles of the input, hidden, and output layers.
- Why would stacked linear layers without nonlinear activations still represent a linear transformation?
- For a dense layer with 7 inputs and 5 neurons, calculate the number of trainable parameters.
- Describe forward propagation using the symbols W, b, z, h, and $\hat{y}$.

Questions 6 to 10

- Explain how a hidden layer helps with the representational limitation revealed by XOR.
- For architecture $4 \rightarrow 8 \rightarrow 3$, calculate the total number of trainable parameters.
- Why does a three-class classifier commonly use three output units?
- Distinguish forward propagation from parameter learning.
- In the Iris MLP, identify which architecture dimensions are determined by the dataset and which are design choices.

Before Week 4

REVIEW

Weighted sum, bias, hidden representation, forward pass, parameter counting.

THINK

Why might sigmoid, tanh, ReLU, and softmax behave differently?

NEXT

Week 4: Activation and Loss Functions.

References and course continuity

Aggarwal, C. C. (2023). Neural Networks and Deep Learning: A Textbook (2nd ed.). Springer. https://doi.org/10.1007/978-3-031-29642-0

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Official online edition: deeplearningbook.org

Week 2 continuity: the Iris ANN preview used Input(4) → Dense(8, ReLU) → Dense(3, softmax). Week 3 explains the architecture and forward computation; activation and loss details follow in Week 4.